

MAGPPIE · VOICE AI

AI Automated Calling System

Technical Plan & Reference Architecture

The engineering blueprint for Magppie's inbound/outbound AI voice operation — the system the Magppie Voice console (already built) controls. Covers architecture, the realtime voice loop, latency budget, the six tool-actions, RAG, compliance, and a phased delivery roadmap.

Magppie Living

SUNROOOF

Ozonetel · NestJS · pgvector

Scope

Backend / voice / data plane (the console is done)

Companion

github.com/jiyasiwach/magppie-calling-bot

Status

Planning · pre-first-call

Prepared

2026 · v1.0 draft

1 • Executive summary

Magppie is building an AI voice agent that handles inbound and outbound phone calls across two brands — Magppie Living (luxury modular kitchens) and SUNROOOF (solar & smart-home) — in Hindi, English and regional languages. It qualifies leads, answers support and order questions, books appointments, and escalates to humans when needed. The already-built Magppie Voice console is its control plane; this document specifies the system behind it.

The design target is a natural, low-latency conversation ("H500–700 ms perceived response on simple turns, 1–2 s on tool-calling turns) with grounded, non-hallucinated answers — every price, warranty status, or order fact comes from a tool call or a retrieved knowledge snippet, never the model's memory.

Guiding principles

- Grounded by default — facts are tool-gated or RAG-sourced; the agent says “let me confirm” rather than guess.
- Latency is a feature — stream tokens into speech; speak filler during tool calls; measure every hop.
- Compliance is not optional — DND/consent/recording checked and logged on every call (India telecom rules).
- One brain, two brands — tenant-isolated agents, data and voices from a single platform.
- Everything observable — each call produces a transcript, tool trace, explainability record and audit log.

2 • The pivotal decision — build vs. buy the voice loop

The riskiest, most latency-sensitive component is the realtime audio loop: voice-activity detection, endpointing, barge-in (letting the caller interrupt), and turn-taking. This single choice sets the timeline and budget.

Option A — Hand-roll in NestJS

Raw Ozonetel media stream wired directly to Deepgram !' LLM !' ElevenLabs inside your own services.

- + **Maximum control, no framework lock-in**
- **You rebuild VAD, endpointing & barge-in**
- **Months of tuning before a natural call**

Option B — Open agent framework &

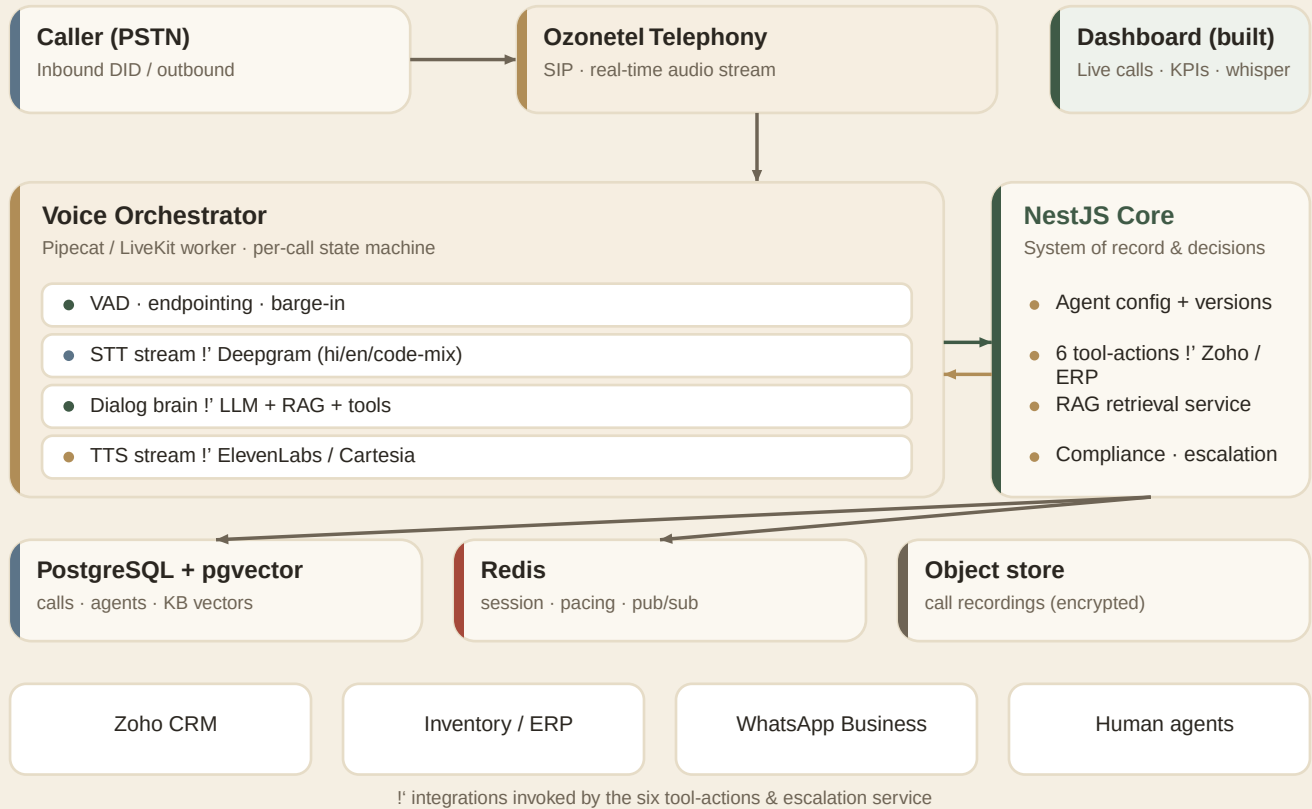
Pipecat or LiveKit Agents runs the audio pipeline; NestJS owns brain, tools, data & control. Ozonetel bridges via SIP.

- + **Turn-taking / barge-in solved out of the box**
- + **Keep your stack; fastest path to first call**
- ~ **One framework dependency to operate**

Recommendation: Option B. The rest of this document assumes it — the framework handles the audio loop; NestJS is the system of record and decision-maker.

3 • System architecture

One diagram, one call. Audio flows through the orchestrator; business logic, data and the six actions live in the NestJS core; the dashboard subscribes to live events.



4 · The conversation loop & latency budget

To feel natural the agent must respond in well under a second on simple turns. The loop streams: speech starts before the full answer is generated, and a short filler (“let me check that...”) covers tool-call time.

Per-turn budget

Stage	Target	Notes
Endpointing (caller stopped)	150–250 ms	VAD in the orchestrator
STT final partial	~100 ms	Deepgram streaming
LLM first token	200–400 ms	Streamed, sentence-chunked
TTS first audio chunk	80–150 ms	Cartesia / ElevenLabs Flash
Perceived response — simple turn	"H 550–700 ms	No external lookup
+ tool call (Zoho/ERP round-trip)	+400–1200 ms	Complex turns land at 1–2 s

Latency levers: co-locate services in an India region, pre-warm model connections, cache hot KB chunks and dealer/warranty lookups, and speak conversational filler the instant a tool call starts.

Barge-in & turn-taking

- Caller can interrupt at any time — TTS stops instantly, STT re-opens; handled by the framework, not hand-coded.
- The Whisper Coach channel injects a supervisor’s text as a system turn mid-call, changing the agent’s very next response.

5 · Subsystems (NestJS modules)

1. Telephony gateway

Ozonetel inbound DIDs per brand, outbound trunk, real-time audio bridge, DTMF, warm transfer to human.

2. Orchestrator worker

The Pipecat/LiveKit pipeline; per-call state machine; barge-in; injects persona + RAG; hosts the Whisper Coach channel.

3. Dialog brain

LLM with strict function-calling for the six actions; per-agent persona/tone/language; guardrail — never quote prices/warranty from memory.

4. RAG knowledge base

Ingest the Drive catalog !' chunk !' multilingual embeddings !' pgvector; retrieve + rerank per turn; log low-confidence answers to the gap detector.

5. Tool-action services

Six typed APIs: create_lead / create_ticket / create_appointment !' Zoho CRM; order_status / warranty_lookup / dealer_lookup !' inventory/ERP. Each returns a trace for explainability.

6. Campaign engine (outbound)

Contact lists, Ozonetel progressive/predictive dialer, Redis pacing controller, predictive ROI + auto-throttle when live ROI drops below threshold.

7. Compliance service

NDNC/DND scrub before every outbound call, consent capture + recording disclosure as the first turn, recording storage, immutable audit log.

8. Escalation & handoff

Rules (sentiment, repeated tool failure, explicit request) !' human transfer, or AI Voice !' WhatsApp !' Human timeline.

9. Realtime gateway

Redis events !' WebSocket !' dashboard (live cards, scrolling transcript, KPI counters). This replaces the mock data layer.

6 • Data model

Postgres schema mirrors the dashboard's types, so going live means turning each mock accessor into a query. Every table carries a tenant column for Magppie Living / SUNROOOF.

agents + agent_versions
git-style history; powers Sandbox & rollback

calls
direction, language, sentiment, outcome, cost

transcripts
per-turn text, timestamp, speaker, sentiment

tool_calls
action, args, result, latency, confidence

campaigns + contacts
pacing, connect/convert, retry state

kb_documents + kb_chunks
source docs + vector chunks (pgvector)

kb_gaps
unanswered questions clustered by topic

customers
360 timeline across voice / WhatsApp / CRM

compliance_logs
consent · DND · recording · escalation

escalations
reason, channel path, handoff timestamps

7 • Safety, compliance & quality

India telecom & consent

- NDNC/DND scrubbing and calling-hour windows before every outbound call; per-brand caller ID.
- Recording disclosure and consent captured as the first turn; opt-out honoured immediately.

Grounding & guardrails

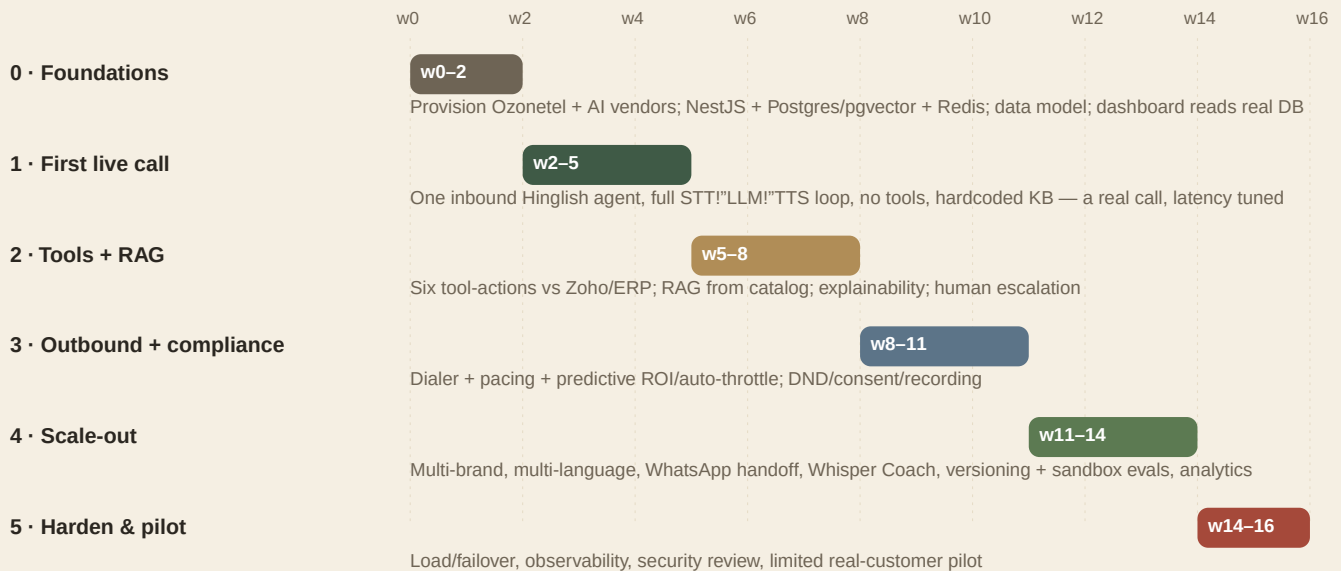
- Tool-gated facts; confidence thresholds trigger “a human will confirm” and escalation.
- PII: masked numbers in UI, encrypted recordings, defined retention policy.

Evals

- The Call Sandbox becomes a regression harness — synthetic-persona suites run before any agent version is published, blocking bad edits.

8 · Phased delivery roadmap

Aggressive but realistic. The milestone that matters most: a real call answered end-to-end by Phase 1 — everything else layers onto a working loop.



Team & top risks

- Team: 1 realtime-voice backend eng · 1 LLM/RAG eng · 1 integrations eng (Ozonetel/Zoho/WhatsApp). Frontend is done.
- Risks: (1) latency on Indian mobile networks · (2) Hindi/code-mixed STT accuracy · (3) hallucinated pricing/warranty !' tool-gate it · (4) telecom compliance · (5) per-minute cost (STT+LLM+TTS+telephony), model early.

9 · How it connects to the console

The Magppie Voice dashboard is already built and running; it is the control plane for this system. Going live is a data-layer swap, not a rewrite.

- `src/lib/mock-data.ts` !' real queries against the Postgres schema above (same return shapes).
- `src/lib/use-live.ts` !' a WebSocket subscription to the realtime gateway (live calls, transcripts, KPI counters).
- The six tool-actions, campaigns, compliance log, KB gaps, Customer 360 and handoff timeline already have their UI — they just need their endpoints.

Blast radius of “go live”: those two files plus the backend. No screen needs rebuilding.

10 · Immediate next steps

- Confirm build-vs-buy (\$2) and lock vendors: telephony (Ozonetel — real-time audio streaming?), STT (Deepgram), LLM, TTS (ElevenLabs vs Cartesia).
- Stand up Phase 0 foundations and wire the dashboard to a real (empty) database.
- Build the Phase 1 voice loop and place the first real end-to-end call — the make-or-break milestone.
- Critical-path dependency: is Ozonetel provisioned with real-time audio streaming today? That gates the first live call.

“Get one real call answered end-to-end first. Tools, campaigns, and polish all layer onto a working loop — a working loop layers onto nothing.”